

SESSION 1

Python Setup, Variables, Data Types & Basic Operations

Modern Data Engineering Course

Week 1 | Module 1: Programming Fundamentals



Instructors: Ameer Ul Islam

What You'll Learn Today



Python Environment Setup

Install Python, set up VS Code, run your first script



Variables & Assignments

Store and manipulate data using Python variables



Core Data Types

Integers, floats, strings, booleans — the building blocks



Basic Operations

Arithmetic, string manipulation, type conversions

Why Python for Data Engineering?

#1 Language

Most used language in
data engineering worldwide

Rich Ecosystem

Pandas, PySpark, dbt,
Airflow — all Python

Easy to Learn

Readable syntax, huge
community, tons of tutorials



Python in the Data Engineering Stack

Data Ingestion (APIs, web scraping) → Transformation (Pandas, PySpark) → Orchestration (Airflow)
→ Quality Checks (Great Expectations)

Setting Up Your Environment

1

Install Python 3.11+

Download from python.org
Check 'Add to PATH'

2

Install VS Code

Download from
code.visualstudio.com
Install Python extension

3

Verify Installation

Open terminal:
`python --version`

 Terminal

```
$ python --version
```

```
Python 3.11.5
```

```
$ python -c "print('Hello, Data Engineer!')"
```

Your First Python Script

 `hello_data.py`

```
# My first Python script

print("Hello, Data Engineer!")

# Basic math

rows_processed = 1500

errors = 23

success_rate = (rows_processed - errors) /
rows_processed

print(f"Success rate: {success_rate:.1%}")
```

Output

```
Hello, Data Engineer!
Success rate: 98.5%
```



Key Takeaway

Python reads top to bottom.
Each line is an instruction.

Variables: Storing Data



Think of variables as labeled containers — you give a name to a piece of data so you can use it later.

```
# Variable assignment

pipeline_name = "daily_sales_etl"

records_count = 50000

is_active = True

error_rate = 0.02


# Reassignment

records_count = 75000 # updated!
```

Naming Rules

- ✓ `user_name` (snake_case)
- ✓ `_private` (underscore start)
- ✓ `count2` (letters + numbers)
- ✗ `2count` (starts with number)
- ✗ `my-var` (has hyphen)
- ✗ `class` (reserved word)

Python Data Types

int

42

Whole numbers
Row counts, IDs

float

3.14

Decimal numbers
Prices, rates

str

"hello"

Text data
Names, paths

bool

True/False

Yes/No values
Flags, status

```
>>> type(42) → <class 'int'>
```

```
type(3.14) → <class 'float'>
```

```
>>> type("hi") → <class 'str'>
```

```
type(True) → <class 'bool'>
```

Numbers: Integers & Floats

Integers (int)

```
# Arithmetic operations
```

```
total_rows = 1000 + 500    # 1500
```

```
batches = 1500 // 400 # 3
```

```
remaining = 1500 % 400    # 300
```

```
squared = 10 ** 2      # 100
```

Floor division vs regular

```
7 / 2  # 3.5 (float!)
```

```
7 // 2 # 3 (int)
```

Floats (float)

```
# Decimal numbers
```

```
price = 29.99
```

```
tax rate = 0.075
```

```
total = price * (1 + tax_rate)
```

32.23925

Rounding

```
round(total, 2)  # 32.24
```

```
Operators:  +  addition      -  subtraction    *  multiply      /  divide      //  floor div    %
modulo      **  power
```

Strings: Working with Text

String Basics

```
# Creating strings

name = "BigQuery"

path = 'data/sales.csv'

# String methods

name.lower()      # "bigquery"
name.upper()      # "BIGQUERY"
name.replace("Big", "Small")

# "SmallQuery"
# f-strings (formatted strings)

db = "BigQuery"
rows = 5000
f"Loaded {rows} rows into {db}"
```

Concatenation

"data" + "_" + "v2" → "data_v2"

Repeat

"_" * 20 → "=====..."

Length

len("BigQuery") → 8

Slicing

"BigQuery"[0:3] → "Big"

Booleans & Comparisons

Comparison Operators

```
# Comparisons return True/False
```

```
10 > 5      # True
```

```
10 == 10    # True
```

```
10 != 5     # True
```

```
10 <= 5     # False
```

```
# Logical operators
```

```
True and False # False
```

```
True or False  # True
```

```
not True       # False
```

Real-World Data Checks

```
# Data quality checks
```

```
row_count = 5000
```

```
null_count = 12
```

```
threshold = 100
```

```
is_valid = row_count > 0
```

```
is_clean = null_count < threshold
```

```
can_proceed = is_valid and is_clean
```

```
print(can_proceed) # True
```



Break Time!

10 Minutes

Try running some variable assignments and `type()` checks in your Python terminal

Type Conversion (Casting)

Data from files and APIs often comes as strings. You need to convert them to the right type!

```
# String to number (very common in data pipelines!)

price_str = "49.99"          # from CSV file
price = float(price_str)      # 49.99 (now a number!)
count_str = "1500"
count = int(count_str)        # 1500

# Number to string (for logging, display)

total = 42000

message = "Processed " + str(total) + " rows"
```

Input & Output

print() — Display Output

```
# Basic print

print("Starting pipeline...")
# Print with variables

rows = 5000

print(f"Loaded {rows} rows")

# Print multiple values

print("Status:", rows, "rows")

# Separator and end

print("a", "b", sep="-") # a-b
```

input() — Get User Input

```
# input() always returns string

name = input("Your name: ")

age = int(input("Your age: "))

print(f"Hi {name}, age {age}!")
```



Pro Tip

Use f-strings for most output — they're cleaner and faster than concatenation!

Conditionals: Making Decisions

Data Pipeline Decision Logic

```
row_count = 5000
null_percentage = 2.5

# if / elif / else

if row_count == 0:
    print("ERROR: No data!")
elif null_percentage > 10:
    print("WARNING: Too many nulls")
elif null_percentage > 5:
    print("NOTICE: Some nulls")
else:
    print("OK: Data looks good!")
```

Indentation Matters

Python uses 4 spaces to define code blocks — no curly braces!

elif = else if

Chain multiple conditions. Only the first True branch runs.

Colon Required

Every if/elif/else line ends with a colon (:)

Loops: Repeating Actions

for Loop

```
# Process a list of tables

tables = ["users", "orders", "products"]
for table in tables:
    print(f>Loading {table}...")
# range() for counting
for i in range(5):
    print(f>Batch {i + 1}")
```

for = known iterations

"Process each table in this list"

while Loop

```
# Retry logic for API calls
attempts = 0
max_retries = 3
success = False
while attempts < max_retries:
    attempts += 1
    print(f>Attempt {attempts}")
    # try connecting...
    success = True
    break
```

while = until condition met

"Keep retrying until it works"



Common Beginner Mistakes

✗ `name = John`

✓ `name = "John"`

Strings need quotes

✗ `"Age: " + 25`

✓ `"Age: " + str(25)`

Can't concat str + int

✗ `if x = 5:`

✓ `if x == 5:`

`==` for comparison, `=` for assignment

✗ `if x > 5
print(x)`

✓ `if x > 5:
 print(x)`

Don't forget the colon and 4-space indent

Session 1 Summary

- ✓ Python environment setup and first script
- ✓ Variables — naming, assignment, reassignment
- ✓ Data types — int, float, str, bool
- ✓ Operators — arithmetic, comparison, logical
- ✓ Type conversion — str(), int(), float()
- ✓ Input/Output — print() and input()
- ✓ Conditionals — if / elif / else
- ✓ Loops — for and while

Up Next: Session 2

Lists, Dictionaries, Functions & File Handling

- Python lists and list operations
- Dictionaries — key-value data storage
- Writing reusable functions
- Reading and writing files (CSV, JSON)

See you next session!